

SourceWise Study Agent

Phase 2 — Backend Development Guide

This document specifies the complete backend architecture for SourceWise, divided into two microservices: a Node.js/Express REST API for authentication, file handling, and data persistence, and a Python FastAPI service for all AI-powered operations including the RAG pipeline, flashcard generation, audio synthesis, and the adaptive study planner engine. Both services share MongoDB Atlas as the primary database.

1. System Architecture

The backend is split into two independently deployable services that communicate via internal HTTP calls. This separation ensures the Node.js API remains lightweight and responsive while long-running AI tasks are handled asynchronously by the Python service.

Service	Port	Responsibility
Node.js API	Port 4000	Auth, file upload, user data, planner CRUD, all REST endpoints for frontend
Python FastAPI	Port 8000	RAG pipeline, embeddings, quiz/flashcard generation, TTS audio, readiness scoring
MongoDB Atlas	Cloud	Primary database. Collections: users, sources, chunks, cards, quizzes, plans, progress
Pinecone / ChromaDB	Cloud / Local	Vector store for document embeddings. Pinecone for production, Chroma for dev
AWS S3 / Cloudinary	Cloud	Stores uploaded PDF/DOCX files and generated MP3 audio files
Redis	Port 6379	Session caching, rate limiting, job queue for async AI tasks (Bull/BullMQ)

2. Node.js / Express API

Step 2.1 — Project Setup & Middleware

- npm init → install express, mongoose, jsonwebtoken, bcryptjs, multer, cors, helmet, express-rate-limit.
- morgan for HTTP request logging, dotenv for environment variables.
- helmet() sets security headers. cors() restricts origins to the React frontend domain.
- express-rate-limit: 100 requests / 15 min per IP on auth routes, 1000 / 15 min on data routes.
- Global error handler middleware catches all unhandled errors and returns consistent JSON responses.

Step 2.2 — Authentication Module

- POST /auth/signup: hash password with bcrypt (saltRounds=12), store user in MongoDB, return JWT.
- POST /auth/login: compare bcrypt hash, issue accessToken (15min) + refreshToken (7 days).
- POST /auth/refresh: verify refreshToken from httpOnly cookie, issue new accessToken.
- POST /auth/logout: clear refreshToken cookie, blacklist token in Redis.
- JWT payload: { userId, email, iat, exp }. Secret stored in JWT_SECRET env variable.
- authMiddleware: extract Bearer token from Authorization header, verify, attach user to req.user.

Step 2.3 — Source Upload & File Management

- Multer middleware configured with diskStorage or memoryStorage depending on file size.
- Allowed MIME types: application/pdf, application/vnd.openxmlformats, text/plain.
- Max file size: 50MB per file, 200MB total per user (configurable).
- After upload: store file in S3/Cloudinary, save metadata to sources collection in MongoDB.
- Enqueue parsing job to Redis/BullMQ queue. Worker picks up job and calls Python /parse endpoint.
- Source status lifecycle: uploaded → parsing → ready → error. Frontend polls GET /sources/:id/status.

Step 2.4 — Planner CRUD Endpoints

- POST /planner/generate: accepts { examDates, hoursPerDay }, forwards to Python planner service, stores result.
- GET /planner/current: returns active weekly plan for authenticated user.
- PATCH /planner/session/:dayId: mark a session as complete, incomplete, or missed.
- POST /planner/catchup: triggers Python engine to redistribute missed sessions.
- GET /planner/streak: returns consecutive days studied, longest streak, total study hours.

Step 2.5 — Progress & Dashboard Endpoints

- POST /progress/quiz: save quiz attempt { sourceId, score, topicBreakdown, duration }.
- GET /dashboard/readiness: aggregates quiz scores, flashcard completion rate, and plan adherence.
- GET /dashboard/activity: returns last 20 activity events for the timeline widget.
- All aggregation queries use MongoDB aggregation pipelines for performance.

3. Python FastAPI — AI Service

The Python service is the intelligence core of SourceWise. It uses LangChain as the orchestration framework connecting OpenAI embeddings, vector storage, and LLM calls. All endpoints are async using Python's asyncio and httpx for non-blocking I/O.

Step 3.1 — FastAPI Setup & Dependencies

- pip install fastapi uvicorn langchain openai pinecone-client chromadb python-docx pdfplumber gtts boto3.
- CORS middleware allowing requests only from the Node.js service (internal) and the frontend domain.
- Pydantic models for all request/response validation. FastAPI auto-generates /docs (Swagger UI).
- Uvicorn with --workers 4 for production. Gunicorn as process manager.

Step 3.2 — Document Parsing Endpoint

- POST /parse: receives file path (from S3) + sourceId. Downloads file, extracts text.
- pdfplumber for PDFs (preserves layout, extracts tables). python-docx for DOCX files.
- Text is split into overlapping chunks: chunk_size=800 tokens, overlap=100 tokens.
- Each chunk stored in MongoDB chunks collection with { sourceId, chunkIndex, text, pageNumber }.
- Chunks vectorized using OpenAI text-embedding-3-small (1536 dimensions) and stored in Pinecone.

Step 3.3 — RAG Query Pipeline

- POST /query: receives { question, sourceIds[], conversationHistory }.
- Embed question → query Pinecone with top-k=6 filter by sourceId → retrieve relevant chunks.
- Build prompt: system instruction + source chunks (as context) + conversation history + user question.
- LLM call: GPT-4o-mini (cost-efficient). Returns answer grounded only in retrieved context.
- Response includes source citations: each sentence can reference the chunk and page it came from.
- LangChain ConversationalRetrievalChain manages multi-turn conversation memory.

Step 3.4 — Flashcard & Quiz Generation

- POST /generate/flashcards: receives { sourceId, topicName, count, difficulty }.
- Prompt instructs GPT-4o to output only JSON: [{question, answer, hint, topic, difficulty}].
- MCQ generation: POST /generate/quiz. Prompt: produce {question, options[4], correctIndex, explanation}.
- Difficulty calibration: Easy = definition-level, Medium = application, Hard = analysis/edge cases.
- Generated cards stored in MongoDB flashcards collection and returned to Node.js API for storage.
- Deduplication check: hash of question text prevents regenerating identical cards.

Step 3.5 — Audio Revision Generator

- POST /audio/generate: receives { topicName, summary, style: 'lecture' | 'podcast' }.
- For lecture style: summary is read verbatim via TTS. For podcast: GPT rewrites it conversationally.
- TTS provider: OpenAI TTS API (alloy voice for neutral, shimmer for conversational). Fallback: gTTS.
- Output: MP3 file uploaded to S3. Returns { audioUrl, duration, transcript }.
- Transcript stored alongside audio for the synchronized transcript panel in the frontend.

- Rate limit: max 10 audio generations per user per day to manage API costs.

Step 3.6 — Adaptive Study Planner Engine

- POST /planner/generate: receives { topics[], examDate, hoursPerDay, startDate }.
- Algorithm: (1) Rank topics by difficulty score + quiz weakness. (2) Apply spaced repetition using SM-2 algorithm. (3) Distribute ranked topics into available days, heaviest topics first.
- SM-2 interval calculation: initial interval = 1 day. After each review, multiply by ease factor (2.5).
- Returns: { days: [{ date, sessions: [{ topic, duration, type: 'learn'|'review'|'quiz' }] }] }.
- POST /planner/catchup: recalculates remaining plan ignoring missed sessions, preserves exam date.
- Readiness formula: $(\text{quiz_avg} * 0.4) + (\text{flashcard_completion} * 0.3) + (\text{plan_adherence} * 0.3)$.

4. MongoDB Collections Schema

Collection	Key Fields
users	_id, name, email, passwordHash, createdAt, settings{}
sources	_id, userId, filename, s3Url, status, mimeType, uploadedAt, topicCount
chunks	_id, sourceId, chunkIndex, text, pageNumber, embeddingId
flashcards	_id, sourceId, userId, question, answer, hint, topic, difficulty, lastReviewed, interval
quizAttempts	_id, userId, sourceId, score, topicBreakdown{}, duration, completedAt
studyPlans	_id, userId, examDate, subject, hoursPerDay, days[], createdAt, status
progress	_id, userId, date, sessionsCompleted, minutesStudied, topicsReviewed[]

5. Environment Variables

```
# Node.js (.env) MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/sourcewise
JWT_SECRET=your_super_secret_key_here REFRESH_TOKEN_SECRET=another_secret
AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... S3_BUCKET_NAME=sourcewise-uploads
PYTHON_SERVICE_URL=http://localhost:8000 REDIS_URL=redis://localhost:6379 # Python
(.env) OPENAI_API_KEY=sk-... PINECONE_API_KEY=... PINECONE_INDEX=sourcewise-embeddings
AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... S3_BUCKET_NAME=sourcewise-uploads
```

6. Backend Deployment Strategy

Component	Platform & Notes
Node.js API	Railway.app — auto-deploy from GitHub. Set env vars in Railway dashboard.
Python FastAPI	Render.com — separate service. Dockerfile with uvicorn. Auto-scale enabled.
MongoDB	MongoDB Atlas M10 cluster (free tier for dev, M10 for prod). Enable IP whitelist.
Pinecone	Pinecone Starter plan (free) for dev. Standard plan for production workloads.
Redis	Railway Redis addon or Upstash (serverless Redis) — free tier covers dev needs.
File Storage	AWS S3 Standard tier. Configure CORS to allow uploads from frontend domain.
CI/CD	GitHub Actions: test → lint → docker build → push → Railway/Render redeploy.